

Cybersecurity Guild

GuildCTF Writeup

September 2026

Name: DhanabalaMurugan	CTFd id: fat>_<watermelon	Roll No: CH25B033
Hostel: Cauvery	Phone: 8122060088	Email: ch25b033@smail.iitm.ac.in

“I’m a hecker and this is my manifesto. I use Arch btw”

`fat>_<watermelon`

Contents

1	Appendix	2
1.1	Resources	2
2	Managerial Stuff	3
2.1	CTFd Username	3
2.2	About You	3
3	Miscellaneous	4
3.1	EAassyy stufff	4
3.2	Sanity Check	4
3.3	Yummy fruit	5
3.4	It’s definitely normal	5
3.5	findmypassword	6
4	Web	8
4.1	Po’s Ramen Shop	8
5	Cryptography	10
5.1	R my SA	10
5.2	XORtune Teller	10
5.3	Exotic* Curves	11
6	Binary Exploitation	12
6.1	GOT a name?	12
6.2	I like Buffets	13
7	GAME	14
7.1	omg it’s migu!!	14

1 Appendix

1.1 Resources

Resources

- [GitHub Repository](#)
- [exploitm writeups](#) — writeup ideas
- [PortSwigger](#) — Web security learning
- [cylabacademy](#) — CTF practice
- [TryHackMe](#) — Cybersecurity learning paths
- [Primer Pdf](#) — Learning concepts
- [over the wire](#) — Learning concepts
- [cyberchef](#) — Learning concepts
- [Linux](#) — Learning linux

2 Managerial Stuff

2.1 CTFd Username

```
~ fat>_<watermelon
```

2.2 About You

a) Tell us about yourself, what makes you tick and why are you interested in Cybersecurity.

- I'm Dhanabala Murugan M, a second year Chemical Engineering student. I really enjoy ricing my Linux setup and distro hopping.
- Locking in for all-nighters with a Red Bull is definitely something that ticks me.
- Cybersecurity interests me because hacking is cool and being a hacker is even cooler (I like living in the terminal).
- Recently, I participated in my first CTF, and it was a really good experience. It made me want to learn more about cybersecurity.

b) What commitments do you have for this and the next semester; How will you balance them with your academic commitments?

- My main commitment apart from academics is Shaastra WebOps.
- I have enough time to spend on the Cybersecurity Guild and contribute consistently.
- I can manage the Guild workload along with my academics by planning my time properly and finishing things according to their priority.

3 Miscellaneous

3.1 EAassyy stuff

Challenge Flag: `exploit{turns_out_krishith_was_wrong}`

Writeup:

Introduction to the problem:

- At first i opened the CTF website, went to the Miscellaneous category and opened the challenge window for EAassyy Stuff.
- The description said that Krishith had bought a really powerful calculator from Alibaba Express which could do anything if you knew how to ask it.
- I started the challenge instance and connected to it from my terminal using netcat with `nc 10.21.232.223 [port]`.
- I first tried some normal inputs like 'yo', 'flag', 'flag please?', 'please', and then some simple maths such as '1 + 2 = 3', '1 + 2', '2 + 1', and '4 + 9 + 8 + 6 + 1' just to see what kind of input it accepted.
- When I entered something like `1 + 2`, it directly evaluated it and returned 3. So this looked very much like Python's `eval()` being used on the input.
- The main issue was that whatever I entered was going straight into a Python interpreter without any proper sanitisation.
- I then tried entering `open`. It returned something like `<built-in function open>`, which confirmed that Python's built-in functions were available.
- That didn't work when I tried `open(flag)`, since Python treated `flag` as a variable instead of a string.
- So I started guessing the filename and tried passing it as a string.
- I first tried `open('flag')` on a fresh connection, but the connection got cut off. I reconnected and tried `open('flag.txt')` instead.
- This time it returned a file object: `<_io.TextIOWrapper name='flag.txt' mode='r' encoding='utf-8'>`.
- From there it was pretty straightforward. I just called `read()` on the file using `open('flag.txt').read()`.
- The expression was evaluated by the backend and the contents of the file, including the flag, were printed back in the terminal.

Solve Script(s)

- *No separate script required; solved interactively via netcat using `open('flag.txt').read()`*

3.2 Sanity Check

Challenge Flag: `exploit{f4k3_Flag}`

Writeup:

- Arigato Gozaimas (Thank You Very Much)

3.3 Yummy fruit

Challenge Flag: `exploit{m{idk}}`

Writeup:

Introduction to the problem:

- At first i downloaded the screenshot from the challenge instance and tried submitting the example flag given in the description. Obviously, that didn't work.
- I looked at the screenshot properly and recognised the old Minecraft textures. That gave me a rough idea of the time period of the video.
- The clue about the author being in 8th standard when watching the video also made me think about Minecraft videos from around 5 or 6 years ago, especially the older creators.
- Looking at the format expected for the flag, it seemed like I had to find a particular series, along with its season details, episode number and timestamp.
- I cropped the image and tried reverse image searches using Yandex Images and Google Lens. They mainly showed similar diamond block optical illusion images from YouTube and TikTok rather than the exact video.
- I also used Gemini to narrow down some of the older Minecraft creators who had proper survival series. This led me to check channels like Skeppy and a few other older creators.
- I then tried TinEye as well. It had a huge indexed database of more than 85.7 billion images, but it still didn't give me an exact match.
- At this point I went back to the actual clues in the challenge. The important ones were the title "Yummy Fruit", the diamond block structure, the red-looking player skin and the subtitle fragment "pretty much, but today i".

That didn't work because manually checking thousands of Minecraft uploads was taking way too much time, but

Then I tried using Google searches with the exact phrases from the screenshot.

- I searched combinations such as "yummy fruit" minecraft to reduce the number of unrelated results.
- This eventually led me to the exact YouTube video: **Minecraft Survival Show - Ep.003 - Yummy Fruit!**, uploaded by **GhostRobo**.
- Still at the end of the day , I couldn't find the screenshot in any of the videos i searched.I'm not sure where the screenshot was taken from.

3.4 It's definitely normal

Challenge Flag: `exploit{moops I forgot}`

Writeup:

Introduction to the problem:

- At first i opened the challenge description for “It’s definitely normal” and downloaded the interview transcript that was provided.
- I read through it and, at first, it just looked like a long and slightly weird interview transcript. But the challenge name made me think there might be something hidden in it.
- While reading it, I noticed a strange sentence where someone was compared to a computer and there was a binary-looking string 10101000101. That made me suspect some kind of binary steganography.
- I opened the transcript in a text editor and noticed that moving the cursor around the text was behaving strangely. It seemed to get stuck between normal characters, which made me suspect invisible Unicode characters.
- After checking this, I found that Zero-Width Space Steganography was being used to hide binary data inside the normal text. Since the characters are invisible, the transcript looks completely normal when you just read it.

That didn’t work because manually trying to copy out invisible characters from such a large text file was basically impossible. The online decoders I tried also messed up the character sequences, but

Then I decided to write a small Python script to extract the hidden characters directly.

- I opened the file using explicit **utf-8** encoding so that the hidden Unicode characters would be preserved.
- I went through the file character by character and picked out the Zero Width Space (`\u200b`) and Zero Width Non-Joiner (`\u200c`) characters.
- I converted those hidden characters into a normal binary string containing only 0s and 1s.
- I split the binary string into groups of 8 bits so that each group could represent one character.
- I converted those bytes back into ASCII characters to reconstruct the hidden message.
- Running the script gave me the complete flag instead of having to manually deal with all the invisible characters.

3.5 findmypassword

Challenge Flag: `exploit{th1_1S_C4l13d_r3v3Rse_Eng1ne3r1ng}`

Writeup:

Introduction to the problem:

- I went to the Miscellaneous category on the CTF website and opened the challenge called ‘findmypassword’.
- The downloaded binary was named `basic_r`. I moved it into my working directory on Arch and renamed it to `basic_rev`.
- I first ran `file basic_rev`, which showed that it was an unstripped 64-bit ELF executable.

- Next I ran `strings basic_rev` to see if there was anything useful stored as plain text inside the binary.
- I also filtered the output using `grep` for words like “flag”, hoping that the actual flag might have been hardcoded somewhere.

That didn’t work because the strings output only showed the normal messages such as “Enter the password:”, “oops , wrong password ;-;” and “Congrats , you got the flag :)”. The actual flag wasn’t present as a normal string.

So I moved on to looking at what the binary was actually doing.

- I tried running it with `ltrace` and `gdb`. I initially ran into some execution issues because of the runtime permissions, but eventually got the binary loaded for analysis.
- I used `objdump -d basic_rev` to look at the assembly, especially the code around the `<main>` function.
- From the assembly I noticed a loop doing XOR operations between the input and the hardcoded `xor_arr` bytes and comparing the result against `enc_arr`.
- I loaded the binary into `gdb` and placed a breakpoint so I could inspect the values while the program was running.
- Using commands such as `x/50xw`, I dumped the values stored in both `xor_arr` and `enc_arr`.
- Once I had both arrays, I reversed the XOR operation character by character and recovered the flag.

4 Web

4.1 Po's Ramen Shop

Challenge Flag: `exploit{d4Mn_Y0u_4r3_f457}`

Writeup:

Introduction to the problem:

- I downloaded `poramenshop.zip`, extracted it, started the challenge instance and opened the website. I also opened the browser developer tools using F12.
- I clicked the redeem code option and got 50 credits, making the balance 150. When I tried the same thing again, the website said that the gift card had already been used.
- Looking at the Network tab, I noticed that the later POST requests to `/api/redeem` were returning a 400 Bad Request.
- The flag cost 1000 credits, while the starting balance was only 100 and one gift card gave 50. So there was no way to reach 1000 just by using the website normally.
- I then went into the extracted files and opened `app.py` to see how the server handled the gift card.
- I also checked the Dockerfile. It contained a fake flag, which was useful for confirming that the challenge setup was working as expected.
- Going back to `app.py`, I noticed that the developer used a mutex lock called `users_lock` inside `get_user()`. That part was protecting access to the global `users` dictionary.
- The interesting part was in the `/api/redeem` route. There was a race condition because the check and the update of the gift card state were separated by a delay.
- In particular, the code had a `time.sleep(0.4)` before the balance was increased and `user["giftcard_used"]` was set to `True`.
- This meant multiple requests could reach the check at almost the same time while `giftcard_used` was still false.

That didn't work when I tried doing it manually through the browser because clicking the button again was far too slow. The server would already set the value to `True` before I could send another request, but

Then I wrote a small multi-threaded Python script to send the requests together.

- I first created a normal request session and obtained the session cookies. This made sure that all the threads were using the same user.
- I created 30 worker threads, matching the maximum allowed by the server's `BoundedSemaphore`.
- I used `threading.Barrier(30)` so that all 30 threads would wait until they were ready before sending their requests.
- Once the barrier released them, all the requests were sent at almost the same time, giving them a chance to reach the 0.4 second delay before the gift card state was updated.

- The terminal output showed that multiple requests were able to add 50 credits. This pushed the balance past the 1000 credit requirement.
- Finally, I used the same session to send a POST request to `/api/buy_ramen`, which returned the actual flag.

Solve Script(s)

- [FLAG solve script](#)

5 Cryptography

5.1 R my SA

Challenge Flag: `exploitm{s0_y0u_kn0w_h0w_t0_f4ct0r153_huh}`

Writeup:

Introduction to the problem:

- The description said “if I give you a such that $a = pq$, how good are you at finding pq ”. Along with the filename, this made me think it was just a normal factorisation challenge.
- I checked the bit length of n . It was 1186 bits, which was a little strange compared to the usual 1024, 2048 or 4096 bit RSA sizes.
- That made me suspect that n might not be made from just two large primes. Instead, it looked like it could be the product of many smaller primes, i.e. multi-prime RSA.
- I first tried Fermat’s factorisation method. It kept running for more than 100M iterations without finding anything useful, so the factors clearly weren’t just two close primes.
- After that I switched to Pollard’s rho, using Brent’s variant. It found a factor in well under a second.
- The factor it found was composite, so I ran Pollard’s rho again on the resulting factors until everything was prime.
- In the end I got 30 distinct primes of roughly 40 bits each, and multiplying them together gave the original n .
- From there I calculated $\phi(n) = \prod(p_i - 1)$, found $d = e^{-1} \bmod \phi(n)$ and decrypted using $m = ct^d \bmod n$. This directly gave the flag.

Solve Script(s)

- [Link To Solve Script1](#)

5.2 XORtune Teller

Challenge Flag: `exploitm{1t_w4s_l1N3er_A1g3br4_a11_A10ng!!!}`

Writeup:

Introduction to the problem:

- I started by reading `chall.py`. The challenge used a custom XORshift32 PRNG. Its `next()` function only used XORs and bit shifts, and the current state was used before moving to the next state.
- The important part here was that XOR and bit shifts are linear over $\text{GF}(2)$. There were no extra constants being added, so the whole `next()` operation could be represented as a fixed 32×32 binary matrix L .

- The integers printed after the encryption also looked suspicious. They weren't random unrelated values. They were the low bytes of five future states, s_{12} through s_{16} .
- So those five leaked bytes gave me 40 linear equations for the 32 unknown bits of s_{12} .
- I solved the equations using Gaussian elimination over $\text{GF}(2)$. The system had rank 32 and gave one unique value for s_{12} . I also checked the result against all five leaked bytes.
- Once I had s_{12} , I used the inverse of L twelve times to go backwards until I reached s_0 .
- This gave me the states used for the original keystream. I generated the 12 blocks, XORed them with the ciphertext and recovered the flag.

5.3 Exotic* Curves

Challenge Flag: `exploit{br0_S4w_P0int_AND_a55umed_3cc}`

Writeup:

Introduction to the problem:

- When I first saw the `Point(x=..., y=...)` class, Alice and Bob's public keys, an IV and an AES-encrypted flag, I assumed it was normal ECDH using a short Weierstrass curve $y^2 = x^3 + ax + b$.
- I even solved for (a, b) using the two public points and tried the leaked value `D` as a possible private scalar.
- That turned out to be the wrong direction. After getting the actual `chall.py`, I found that the "point addition" wasn't really elliptic curve addition at all.
- It was actually multiplication in the ring $Z_p[\sqrt{D}]$:

$$(P_x, P_y) \cdot (Q_x, Q_y) = (P_x Q_x + D P_y Q_y, P_x Q_y + P_y Q_x).$$

- So the challenge was basically disguising field multiplication as point addition and making it look like an elliptic curve problem.
- I checked that `D` was a quadratic non-residue modulo `p`. This means the construction gives the field F_{p^2} . I also checked that `G`, `A` and `B` all had norm 1, putting them in the subgroup of order $p + 1$ rather than the full group of order $p^2 - 1$.
- I then factored $p + 1$. Most of the factors were small and smooth, with powers of primes up to 41 and a few medium-sized factors. There was only one remaining prime of about 59 bits.
- That made Pohlig–Hellman the obvious approach.
- I ran Pohlig–Hellman for each prime-power factor of $p + 1$, using BSGS for the smaller factors and Pollard's rho for the 59-bit factor.
- After CRT-combining the results, I recovered Bob's private key n_b modulo $\text{ord}(G)$. I then calculated the shared secret A^{n_b} , derived the AES key using `sha1(str(shared_x))[16]`, and decrypted the ciphertext using the given IV.

6 Binary Exploitation

6.1 GOT a name?

Challenge Flag: `exploit{th6t_W65n'T_y0Ur_nAm3!!}`

Writeup:

Introduction to the problem:

- I went to the Pwn category on the challenge portal and opened the challenge called ‘GOT a name?’.
- I downloaded the binary `chal`. The hint said “double checking is a good practice, or is it??”, which made me think there could be some kind of double lookup or missing verification involved.
- I opened my Arch terminal and connected to the remote service using `nc 10.21.232.223 57246`.
- I also ran `objdump -d chal | grep -A 15 "<gadget>:"` to check the binary for useful gadgets or pre-existing functions.
- I checked the binary protections using my `got.py` setup.
- The important result was that RELRO was disabled and PIE was also disabled. Since PIE was off, the binary was loaded at the predictable address `0x400000`, and with no RELRO the GOT remained writable.

A normal stack overflow wasn’t a good option because the binary had a stack canary enabled. Overwriting the stack in the usual way would hit the canary and the program would abort before I could redirect execution.

So I decided to work with the GOT instead.

- I set up `got.py` to communicate with the remote challenge instance.
- I obtained a pointer leak from the program output, which helped with the memory addresses I needed.
- I then worked out the address of a standard library function in the GOT.
- Using the available write primitive, I constructed a payload that targeted the required GOT entry and replaced its function pointer with the address of the shell gadget.
- After triggering the overwritten function, execution went to the gadget instead.
- This gave me an interactive shell on the remote instance, and I used `cat flag.txt` to get the flag.

6.2 I like Buffets

Challenge Flag:

Writeup:

Introduction to the problem:

- I went back to the Pwn section on the scoreboard and opened the challenge called ‘I like Buffets’.
- The challenge came with two files, `chal` and `chal.c`. I downloaded both of them.
- The description said “i cant miss my sunday buffet”, so the word buffet immediately made me suspect a buffer overflow.
- I started the remote instance and connected using `nc 10.21.232.223 51217`.
- I opened `chal.c` and checked how the variables were arranged on the stack.
- The `Username` variable was a 300-byte buffer.
- More importantly, the input was being taken using `scanf("%s", Username)`. Since there was no length limit, the input could be much larger than the buffer.

That didn’t work with a normal short username because the generated `session_id` was just a random value between 0 and 15. There was basically no chance of it matching the large negative number used by the program for verification, but

Then I checked the stack layout and tried overwriting the value directly.

- I found that the 64-bit `session_id` was stored immediately after the username buffer on the stack.
- The value the program was checking against was `-60452921414551197`.
- I made a payload with 308 bytes of padding to reach the target value.
- I converted the target value to the required little-endian byte representation and appended it after the padding.
- I sent the complete payload to the program.
- The overflow changed the value of the session variable to the required value, which made the check succeed and reached the `print_flag()` function.

7 GAME

7.1 omg it's migu!!

Challenge Flag: `exploit{16_x3N3_shouldnt_get_it_WX7dG0Q}`

Writeup:

Introduction to the problem:

At first I tried...

I identified that the program is vulnerable because...

That didn't work because..., but

Then I tried to...

...

Solve Script(s)

[Link To Solve Script1](#)